

PYTHON

Introduction to Programming & Python Fundamentals

From core programming concepts to functions, loops, data structures and recursion



Introduction to Programming Concepts

- Program, Programmer & Programming Language
- Types of Programming Languages
- How Programming Works



What is a Program?

- A program is a set of step-by-step instructions written to tell a computer exactly what to do.
- It is created by a human (a programmer) and written in a language the computer can understand.
- Examples: a calculator app, a web browser, a mobile game — all are programs.
- Without programs, computer hardware cannot perform any useful task.

What is a Programmer?

- A programmer (or developer/coder) is a person who writes, tests, and maintains computer programs.
- Programmers translate real-world problems into logical steps a computer can execute.
- Core skills: problem solving, logical thinking, and knowledge of one or more programming languages.
- Programmers also debug (fix errors) and improve existing code.

What is a Programming Language?

- A programming language is a formal set of rules and syntax used to write instructions for a computer.
- It acts as a bridge between human thinking and machine-executable code.
- Each language has its own grammar (syntax) and meaning (semantics).
- Examples: Python, Java, C++, JavaScript, C.

Types of Programming Languages

High-Level Languages

- Close to human language, easy to read/write
- Examples: Python, Java, C++, JavaScript

Low-Level Languages

- Close to machine/hardware, harder to read
- Examples: Machine Language, Assembly Language

By Execution

- Compiled languages (C, C++) and Interpreted languages (Python, JavaScript)

Types of Low-Level Languages

Machine Language

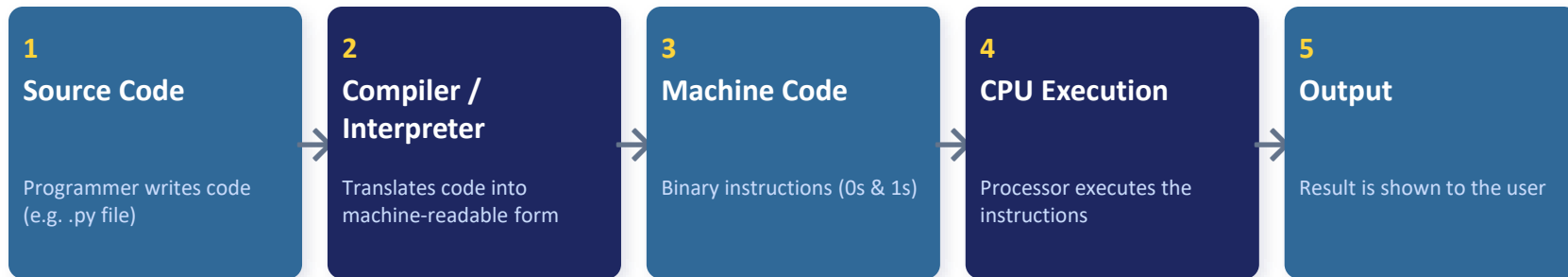
- Written in binary (0s and 1s) — the only language a CPU directly understands
- Extremely fast but very difficult for humans to read or write

Assembly Language

- Uses short mnemonic codes (e.g., MOV, ADD, SUB) instead of raw binary
- Converted into machine code using an Assembler
- Still hardware-specific but easier for humans than pure binary

How Programming Works

From source code to a running program — the journey your code takes:



Compiled languages (e.g. C, C++) translate the entire program into machine code before running it. **Interpreted languages** (e.g. Python) translate and execute code line-by-line at runtime.

Introduction to Programming — Multiple Choice Questions

1. A program is best described as:

- A) A set of instructions for a computer B) A type of hardware C) A file extension D) A physical device

2. Which is a low-level language?

- A) Python B) Assembly Language C) JavaScript D) Java

3. Machine language is written using:

- A) English words B) 0s and 1s C) Mnemonics D) Symbols only

4. Python code is executed by a/an:

- A) Assembler B) Interpreter C) Linker D) Compiler only

5. Who converts a real-world problem into code?

- A) Programmer B) Printer C) Monitor D) Router

Answer Key: 1-A 2-B 3-B 4-B 5-A

What is Python?

- Python is a high-level, general-purpose, interpreted programming language created by Guido van Rossum (first released 1991).
- Python was named after the British comedy TV show *Monty Python's Flying Circus*.
- Known for simple, readable syntax that resembles plain English.
- It is widely used for web development, data science, artificial intelligence (AI), machine learning, automation, game development, desktop applications, and cybersecurity.
- Python is dynamically typed and supports multiple programming styles (procedural, object-oriented, functional).
- Python is a case sensitive language.



Why We Use Python?

- Easy to learn and read — beginner friendly syntax.
- Huge standard library and third-party packages (NumPy, Pandas, Django, TensorFlow...).
- Cross-platform — runs on Windows, macOS, and Linux.
- Large, active community and strong industry demand.
- Great for rapid prototyping and automation of repetitive tasks.

1

Python Basics

- Installation & VS Code setup
- `print()`, Escape Sequence, variables & data types
- Input/Output, Split, comments, String & operators

Python Basics: Installation

Steps to Install Python

- Go to the official website: python.org/downloads
- Download the latest stable version for your OS (Windows/macOS/Linux)
- Run the installer — check “Add Python to PATH” before installing (Windows)
- Verify installation by opening a terminal and typing: `python --version`

Official Website Link

- <https://www.python.org/downloads/>

Setting Up a Project Folder in VS Code

Creating a Folder for Your Project

- Install Visual Studio Code from code.visualstudio.com
- Install the official “Python” extension from the VS Code Extensions panel
- Open VS Code → File → Open Folder → create/select a new folder (e.g. “my_python_project”)
- Open the Terminal (Ctrl + `) inside VS Code and create a file: e.g. main.py
- Right-click the file → Run Python File, or press the ▶ Run button

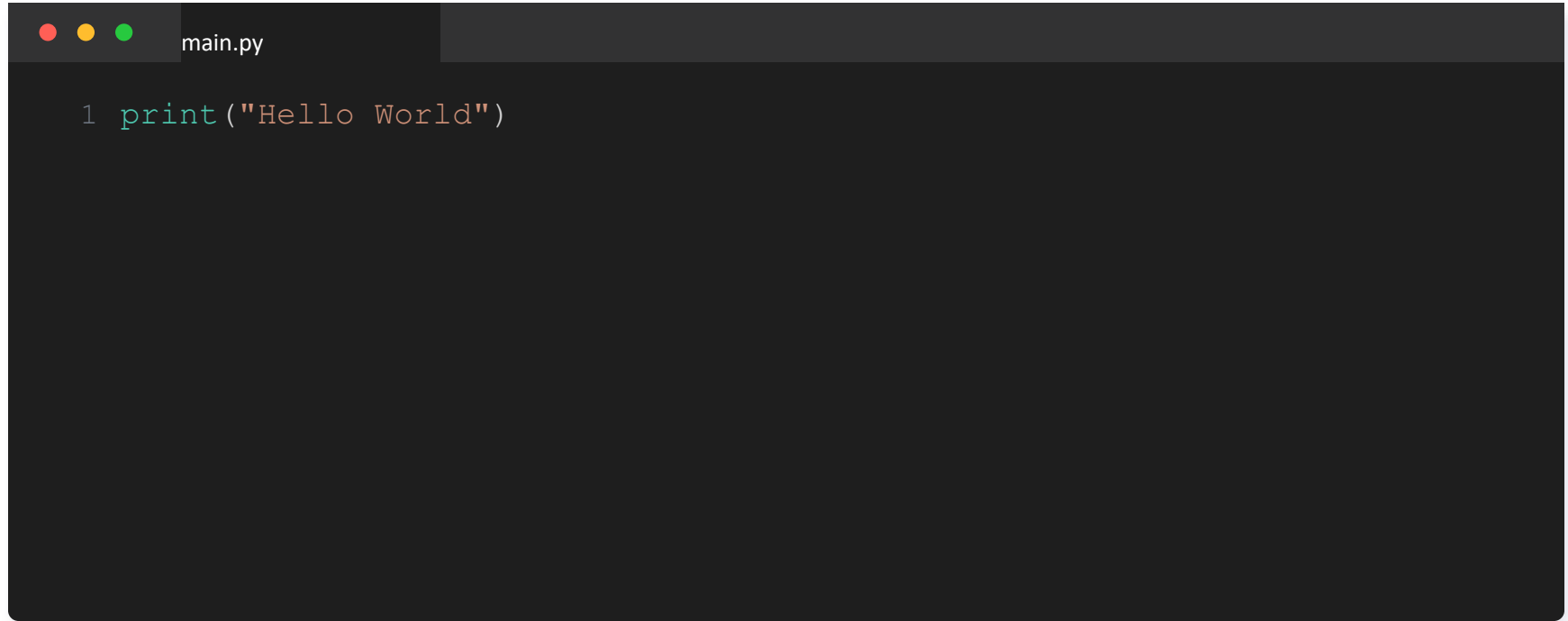


print() Function

- The print() function is used to display output on the screen.
- It is the first function that beginners learn in Python.
- Syntax: `print("Text")`

print() Function — Example

print() displays whatever text or value is passed inside the parentheses.

A code editor window with a dark background. The title bar at the top shows three colored circles (red, yellow, green) and the text 'main.py'. The code area contains a single line:

```
1 print("Hello World")
```

 The text is color-coded: '1' is light blue, 'print' is green, and the string is orange.

Output / Note: Hello World

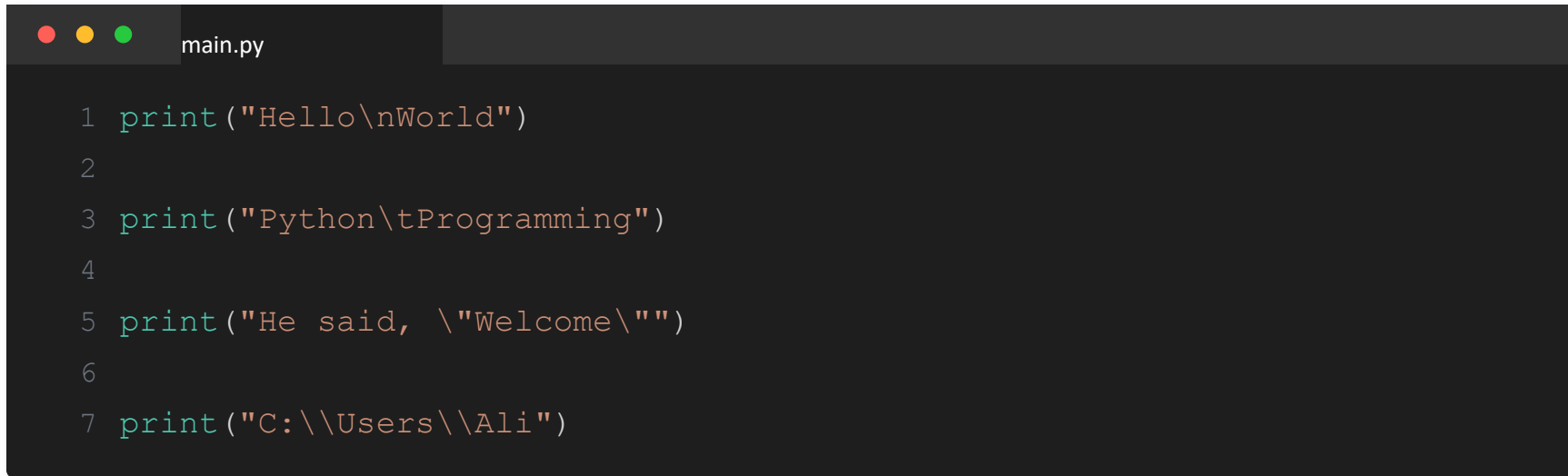
Escape Sequences

- Escape sequences are special characters that begin with a backslash (\) and are used to perform special formatting inside strings.

Escape Sequence	Meaning
\n	New Line
\t	Horizontal Tab
\\	Backslash
\"	Double Quote
\'	Single Quote

Escape Sequences — Examples

Common uses of escape sequences inside strings.



```
1 print("Hello\nWorld")
2
3 print("Python\tProgramming")
4
5 print("He said, \"Welcome\")
6
7 print("C:\\Users\\Ali")
```

Output / Note:

- Hello
World
- Python Programming
- He said, "Welcome"
- C:\Users\Ali



Advantages of Escape Sequences

- Makes output neat and readable.
- Helps format text.
- Prints special characters correctly.
- Used in file paths and formatted messages.

Variables in Python

- A variable is a named storage location in memory that is used to store data.
- It allows us to save information and use it later in a program.
- Think of a variable as a container (box) that holds a value — the value can be changed whenever needed.

Variable	Value
name	Ali
age	20
marks	85



Advantages of Variables

- Store data easily.
- Make programs reusable.
- Reduce repetition.
- Improve code readability.

Rules for Naming Variables

Python has some rules for naming variables:

- Variable names must start with a letter (A-Z or a-z) or an underscore (_).
- They cannot start with a number.
- Spaces are not allowed.
- Special characters such as @, #, %, & are not allowed.
- Variable names are case-sensitive.

Correct vs Incorrect Variable Names

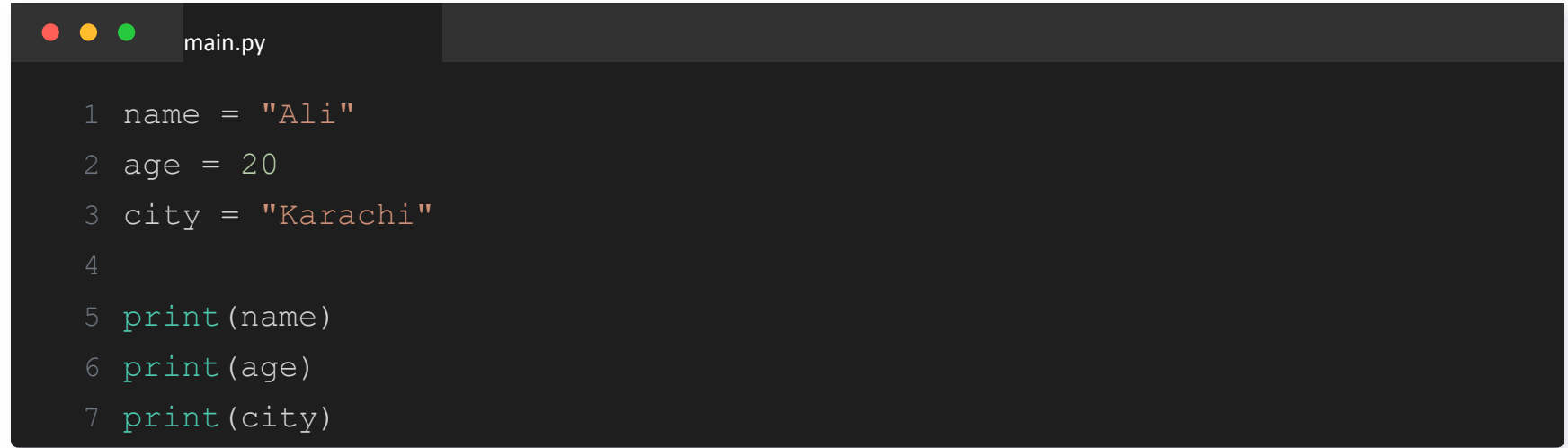
Following naming rules avoids errors when creating variables.

```
main.py

1  # Correct Variable Names
2  name = "Ali"
3  age = 20
4  _marks = 90
5  studentName = "Ahmed"
6
7  # Incorrect Variable Names
8  # 2name = "Ali"           -> starts with a number
9  # student name = "Ali"  -> contains a space
10 # @name = "Ali"         -> special character
```

Assigning Values to Variables

The assignment operator (=) is used to assign a value to a variable.



```
1 name = "Ali"
2 age = 20
3 city = "Karachi"
4
5 print(name)
6 print(age)
7 print(city)
```

Output / Note:

- Ali
- 20
- Karachi

Multiple Variable Assignment

Python allows assigning values to multiple variables in a single line.

main.py

```
1 x, y, z = 10, 20, 30
2 print(x)
3 print(y)
4 print(z)
5
6 a = b = c = 100    # same value to all
7 print(a)
8 print(b)
9 print(c)
```

Output / Note:

- 10
- 20
- 30
- 100
- 100
- 100

Changing Variable Values

A variable's value can be changed at any time during program execution.

main.py

```
1 name = "Ali"  
2 print(name)  
3  
4 name = "Ahmed"  
5 print(name)
```

Output / Note:

- Ali
- Ahmed

Best Practices for Variables

Use meaningful, descriptive names — they make code easier to read and understand.

main.py

```
1  # Good
2  student_name = "Ali"
3  total_marks = 500
4
5  # Bad
6  a = "Ali"
7  b = 500
```

Data Types in Python

- A data type defines the type of value stored in a variable. Python automatically identifies the data type based on the assigned value.

Data Type	Description	Example
int	Integer numbers	10, 50, -5
float	Decimal numbers	3.14, 10.5
str	Text / String	"Hello"
bool	True or False	True
list	Ordered collection	[10, 20, 30]
tuple	Ordered immutable collection	(1, 2, 3)
set	Unordered unique collection	{1, 2, 3}
dict	Key-value pairs	{"name": "Ali"}

Data Types — Example

Python automatically identifies a variable's data type. Use `type()` to check it.

main.py

```
1 age = 20
2 price = 99.99
3 name = "Ali"
4 is_student = True
5
6 print(type(age))
7 print(type(price))
8 print(type(name))
9 print(type(is_student))
```

Output / Note:

- `<class 'int'>`
- `<class 'float'>`
- `<class 'str'>`
- `<class 'bool'>`

Type Conversion

Python allows converting one data type into another using `datatype(value)`.

main.py

```
1 age = "20"
2 print(type(age))
3
4 age = int(age)
5 print(type(age))
6
7 age = float(age)
8 print(type(age))
9
10 age = str(age)
11 print(type(age))
```

Output / Note:

- `<class 'str'>`
- `<class 'int'>`
- `<class 'float'>`
- `<class 'str'>`

Python Print Formatting

Three common ways to display variables inside the `print()` function.

- Python provides three common ways to display variables inside the `print()` function.
- Using Comma `(,)` — the simplest way to print multiple values together.
- Using f-String `f"""` — the modern, recommended way to insert variables into text.
- Using `.format()` — a flexible method using placeholders `{}`.

Using Comma (,) in print()

What is Comma (,) in Python?

- The comma is used inside the print() function to display multiple values.
- It does not concatenate strings — it prints each value separately.
- Python automatically inserts a space between the values.
- Syntax: `print(value1, value2)`

Comma — Examples

Printing multiple variables together using a comma.

main.py

```
1 first_name = "Ali"
2 last_name = "Khan"
3 print(first_name, last_name)
4
5 name = "Ali"
6 age = 20
7 print("Name:", name)
8 print("Age:", age)
```

Output / Note:

- Ali Khan
- Name: Ali
- Age: 20



Advantages of Using Comma

- Prints multiple values together.
- Automatically adds spaces between values.
- Easy to use with variables and strings.
- Commonly used for displaying output.

f-String (f""")

A modern way to insert variables and expressions directly into a string.

- **Definition**
- An f-string (Formatted String Literal) is a modern way to insert variables and expressions directly into a string. Introduced in Python 3.6 — write f before the string and place variables inside curly braces {}.
-
- **Why is it Used?**
- To insert variables directly into a string.
- To make the code cleaner and easier to read.
- To perform calculations inside the string.
- To write modern and professional Python code.

f-String — Syntax & Example

main.py

```
1 # Syntax: print(f"Text {variable}")
2
3 name = "Ali"
4 age = 20
5
6 print(f"My name is {name}.")
7 print(f"I am {age} years old.")
```

Output / Note:

- My name is Ali.
- I am 20 years old.

f-String — Advantages

Why f-strings are the preferred choice in modern Python.

- Easy to read and understand.
- Short and clean syntax.
- Faster than `.format()`.
- Can perform expressions inside `{}`.
- Most recommended method in modern Python.

.format() Method

Inserting variables into a string by replacing placeholders {}.

- **Definition**
- The .format() method is used to insert variables into a string by replacing placeholders {} with values. The values are passed inside the .format() function.
-
- **Why is it Used?**
- To insert variables into a string.
- To format strings in a structured way.
- To work with older Python versions.
- To control the position and names of inserted values.

.format() — Syntax & Example

main.py

```
1 # Syntax: print("Text {}".format(variable))
2
3 name = "Ali"
4 age = 20
5
6 print("My name is {}".format(name))
7 print("I am {} years old.".format(age))
```

Output / Note:

- My name is Ali.
- I am 20 years old.

.format() — Advantages

Why .format() is still useful, especially in older Python code.

- Supported in older Python versions.
- Can use positional indexes ({0}, {1}).
- Can use named arguments ({name}, {age}).
- Useful for formatting complex strings.
- Easy to rearrange the order of variables.

Comparison Table

Comma, f-String, and .format() side by side.

Method	Syntax	Python Version	Recommended
Comma ,	<code>print("Name:", name)</code>	All versions	★ Good
f-String	<code>print(f"Name: {name}")</code>	Python 3.6+	★ ★ ★ Best
.format()	<code>print("Name: {}".format(name))</code>	All versions	★ ★ Good

Which Method Should You Use?

Choosing the right print formatting method for your situation.

Situation	Recommended Method
Beginner Programs	Comma ,
Modern Python Projects	f-String
Older Python Code	.format()

String Concatenation Using (+)

What is String Concatenation?

- String concatenation is the process of joining two or more strings into a single string.
- In Python, the plus (+) operator is used to concatenate strings.
- Syntax: `string1 + string2`

String Concatenation — Examples

Joining strings together using the + operator.

main.py

```
1 first_name = "Ali"
2 last_name = "Khan"
3 full_name = first_name + " " + last_name
4 print(full_name)
5
6 city = "Karachi"
7 print("City: " + city)
```

Output / Note:

- Ali Khan
- City: Karachi



Advantages of Concatenation

- Joins two or more strings.
- Creates a new string.
- Makes output more meaningful.
- Commonly used to display messages.

Difference: (+) vs Comma (,)

+ Operator	Comma (,)
Used for string concatenation.	Used to print multiple values.
Joins strings into one string.	Does not join strings.
Creates a new string.	Displays values separately with spaces.
Works only with compatible data types (or requires conversion).	Can print different data types together without conversion.

- Use + when you want to join strings into a single string.
- Use , when you want to print multiple values without converting their data types.
- For modern Python, f-strings are generally the most recommended way to format output.
- Note: + with a string and an integer raises an error unless converted with str().

Choosing the Right Method

A quick reference for when to use each printing/formatting technique.

Method	When to Use
, (Comma)	When printing simple values or multiple variables.
+ (Concatenation)	When joining only string values together.
f-string	Best choice for modern Python. It is fast, readable, and recommended.
.format()	Useful when working with older Python code or when advanced string formatting is needed.

Input & Output in Python

- Input is the process of receiving data from the user; output is the process of displaying information on the screen.
- Python provides built-in functions to perform input and output operations.
- `input()` → used to take input from the user.
- `print()` → used to display output on the screen.

Input in Python

`input()` displays a message, the user enters a value, and it is stored in a variable.

main.py

```
1 name = input("Enter your name: ")
2 print(name)
```

Output / Note:

- Enter your name: Ali
- Ali

Taking Multiple Inputs

You can call `input()` multiple times to collect several values from the user.

main.py

```
1 name = input("Enter your name: ")
2 city = input("Enter your city: ")
3
4 print("Name:", name)
5 print("City:", city)
```

Output / Note:

- Enter your name: Ali
- Enter your city: Karachi
- Name: Ali
- City: Karachi

Integer Input

`input()` always returns a string — use `int()` to convert it for math operations.

main.py

```
1 age = int(input("Enter your age: "))  
2 print("Age:", age)
```

Output / Note:

- Enter your age: 20
- Age: 20

Float Input

Use `float()` to accept decimal numbers from the user.

main.py

```
1 price = float(input("Enter product price: "))  
2 print("Price:", price)
```

Output / Note:

- Enter product price: 99.99
- Price: 99.99



Advantages of Input

- Allows users to enter their own data.
- Makes programs interactive.
- Accepts different types of information.

split() Method in Python

- The split() method divides a string into multiple parts based on a specified separator.
- By default, it splits the string at spaces and returns the result as a list.
- Commonly used with input() to accept multiple values in a single line.
- Syntax: `string.split(separator)` or `input().split()`

split() — Splitting a Sentence

split() separates a sentence into a list of words at each space.

main.py

```
1 text = "Python is easy"
2 words = text.split()
3 print(words)
```

Output / Note: ['Python', 'is', 'easy']

split() with User Input

Use split() to accept two values from the user on a single line.

main.py

```
1 name, city = input("Enter your name and city: ").split()
2
3 print("Name:", name)
4 print("City:", city)
```

Output / Note:

- Enter your name and city: Ali Karachi
- Name: Ali
- City: Karachi

Comments in Python

What are Comments?

- Comments are explanatory notes written inside a Python program.
- They describe the purpose of code, improve readability, and make programs easier to understand.
- Python ignores comments during execution — they do not affect the output.

Why Do We Use Comments?

- Explain the purpose of the code.
- Improve code readability, debugging and maintenance.
- Help other programmers understand the program.

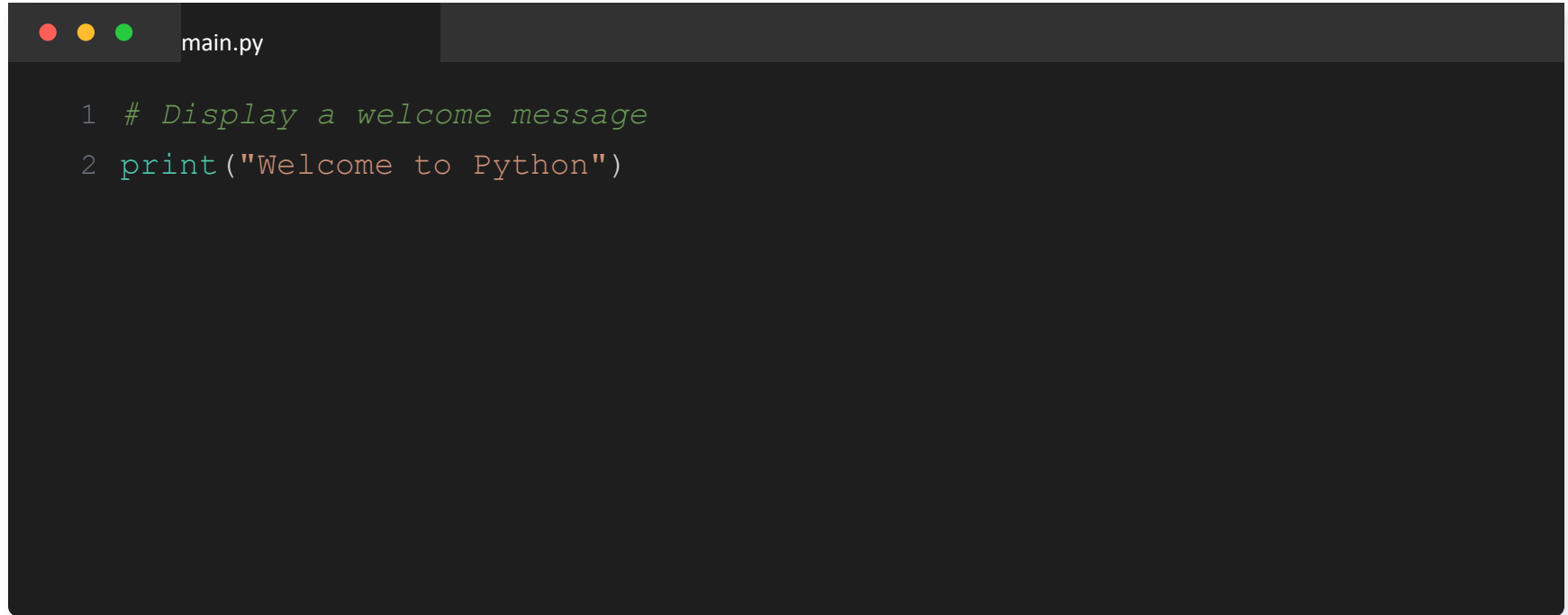
Types of Comments

- Python supports two types of comments:

Comment Type	Description	Symbol
Single-Line Comment	Used for a single line of explanation.	#
Multi-Line Comment	Used for multiple lines of explanation or documentation.	''' ''' or """ """

Single-Line Comment

Starts with the # symbol — Python ignores everything after it on that line.

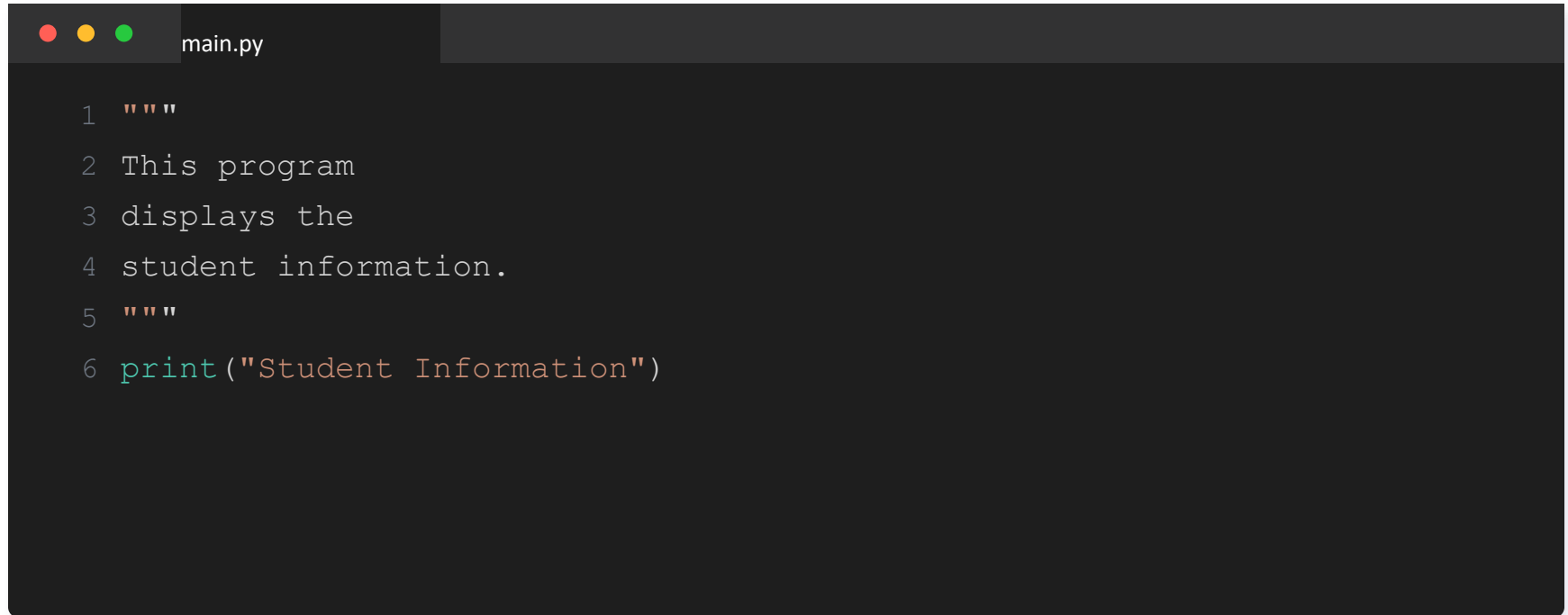


```
1 # Display a welcome message
2 print("Welcome to Python")
```

Output / Note: Welcome to Python

Multi-Line Comment

Triple quotes are commonly used to write explanations across multiple lines.

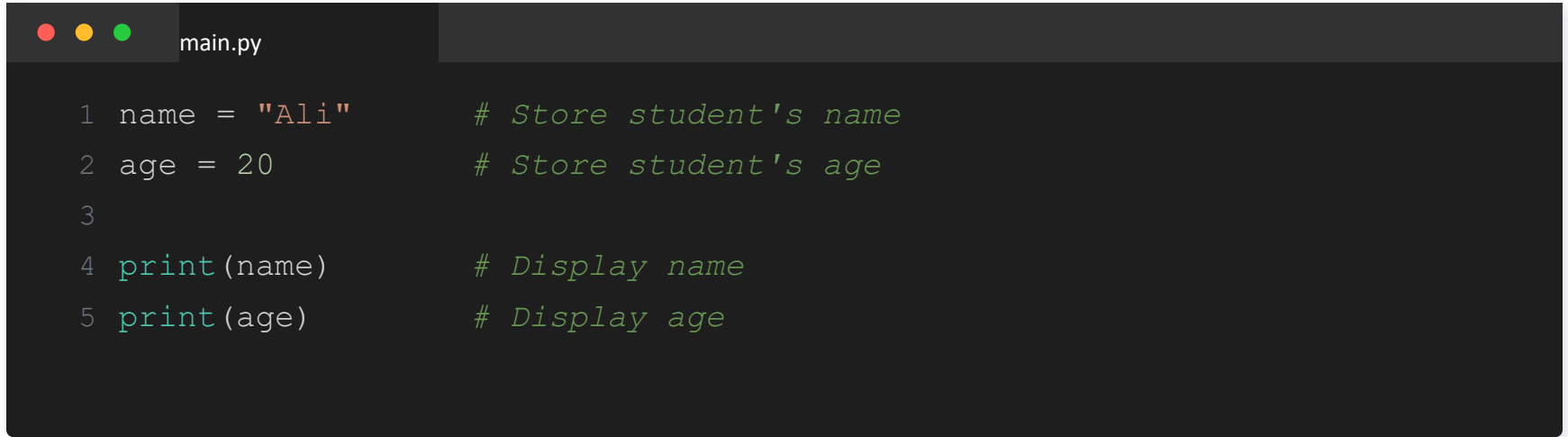


```
1 """
2 This program
3 displays the
4 student information.
5 """
6 print("Student Information")
```

Output / Note: Student Information

Comments Inside Code (Inline)

Comments can also be written at the end of a line of code.



```
1 name = "Ali"      # Store student's name
2 age = 20           # Store student's age
3
4 print(name)        # Display name
5 print(age)         # Display age
```

Output / Note:

- Ali
- 20



Advantages of Comments

- Improve code readability.
- Explain complex logic.
- Help during debugging.
- Make teamwork easier.
- Improve code maintenance.
- Useful for documentation.

What is a String?

- A string is a sequence of characters enclosed inside quotes.
- Strings can contain: alphabets, numbers, symbols, and spaces.
- Example: `name = "Ali Ahmed"` → `"Ali Ahmed"` is a String.

Types of Quotes in Strings

Python allows us to create strings using:

- 1. Single Quotes — `language = 'Python'`
- 2. Double Quotes — `language = "Python"`
- 3. Triple Quotes — used for multiline strings

Why Different Quotes Are Used?

- Mostly developers use double quotes because they are more common and readable.
- Sometimes we use different quotes depending on the situation.

Quotes — Examples

The same string can be created with single, double, or triple quotes.

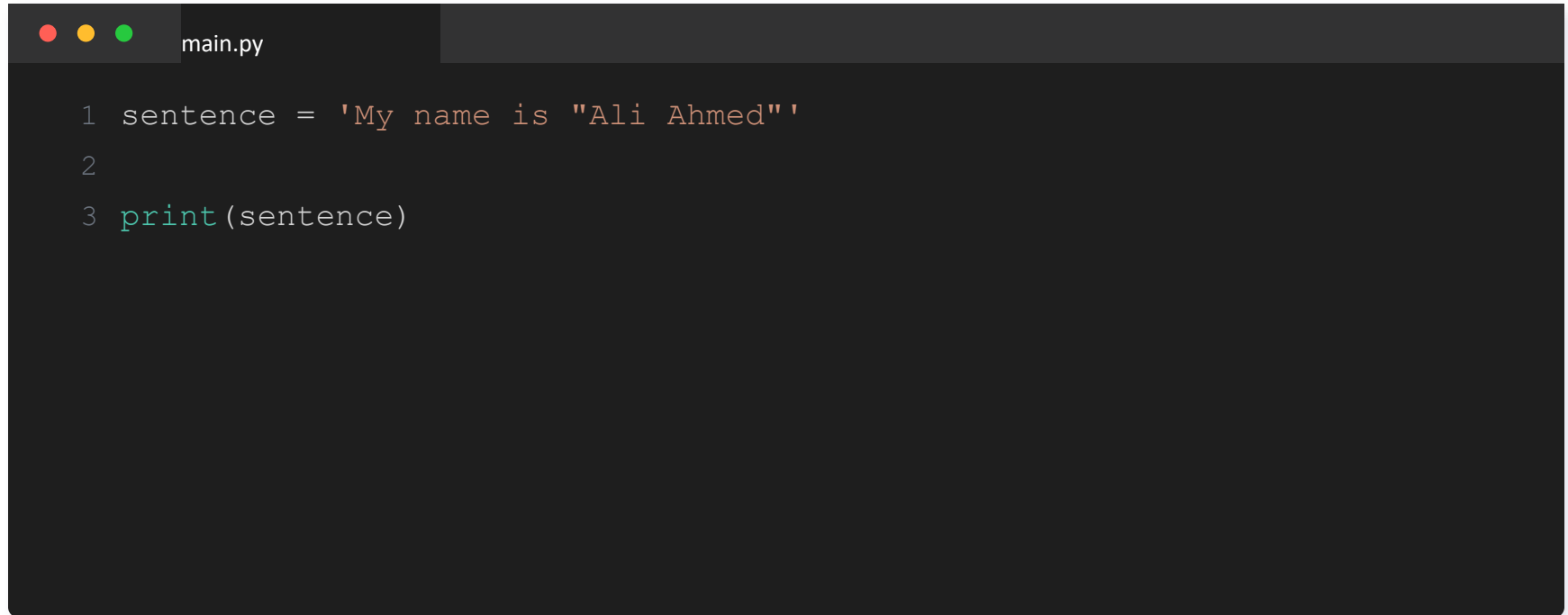


main.py

```
1 language = 'Python'           # single quotes
2 language = "Python"          # double quotes
3
4 message = """Welcome
5 to
6 Python"""                     # triple quotes
```

Using Single Quotes

Single quotes are useful when the string itself contains double quotes.

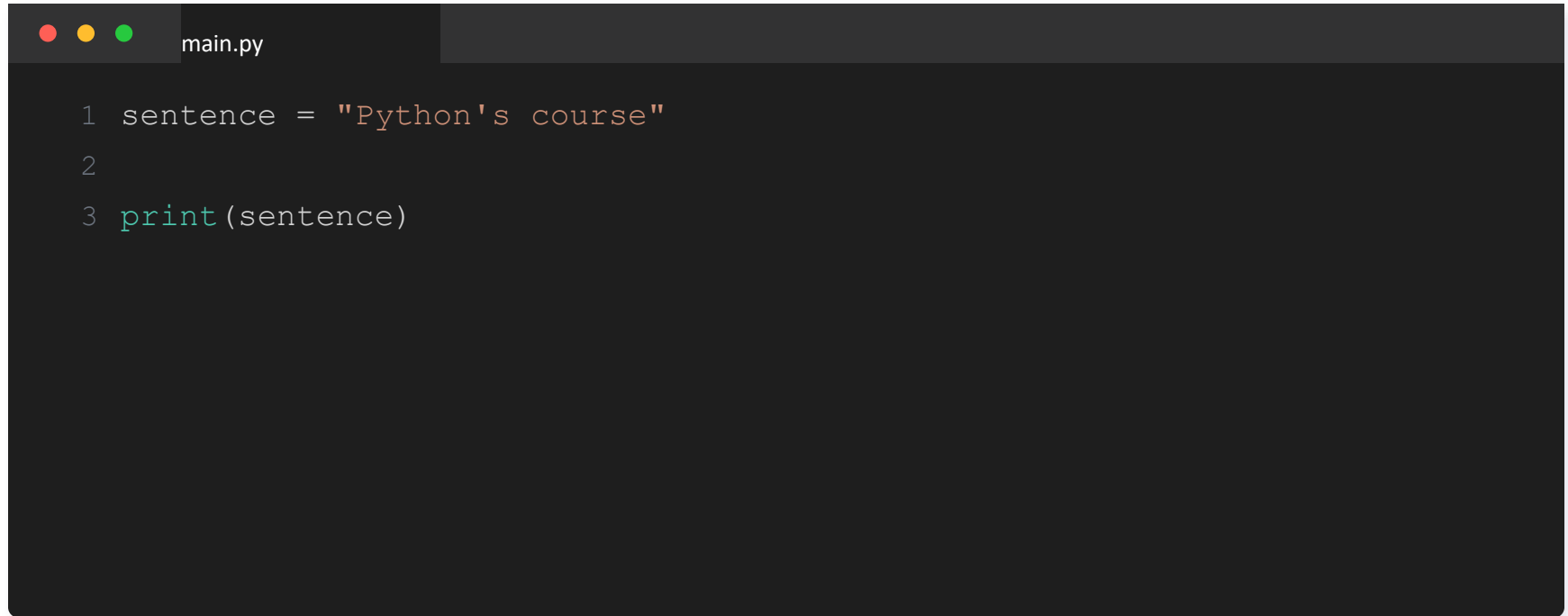


```
1 sentence = 'My name is "Ali Ahmed" '
2
3 print(sentence)
```

Output / Note: My name is "Ali Ahmed"

Using Double Quotes

Double quotes are useful when the string contains a single quote (apostrophe).

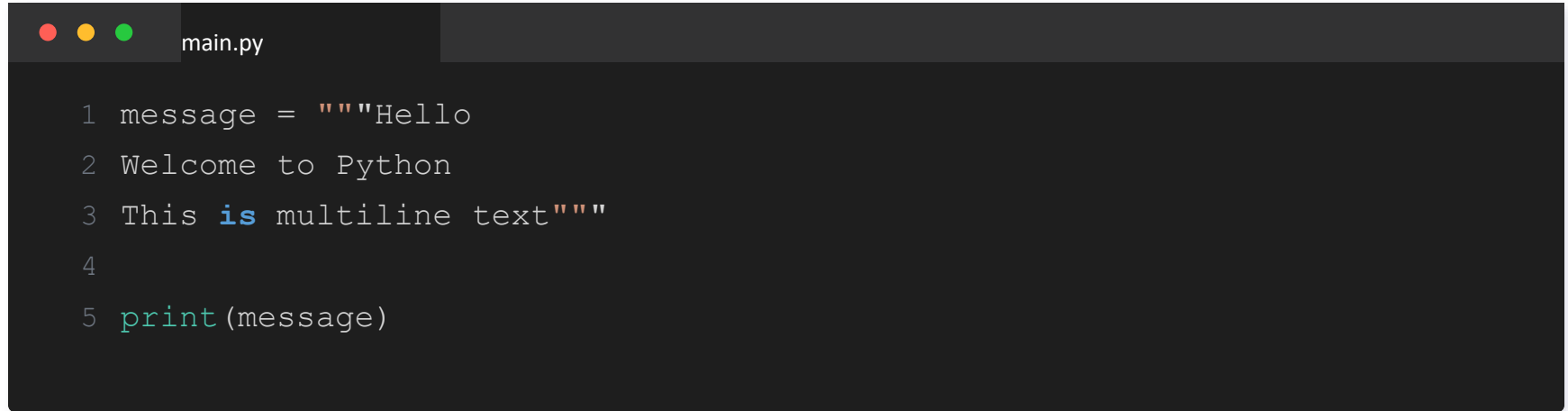


```
1 sentence = "Python's course"
2
3 print(sentence)
```

Output / Note: Python's course

Using Triple Quotes

Triple quotes are used to write multiline strings.



```
1 message = '''Hello
2 Welcome to Python
3 This is multiline text'''
4
5 print(message)
```

Output / Note:

Hello
Welcome to Python
This is multiline text



String Operations

- String operations are used to work with strings.
- 1. Concatenation — joining two or more strings
- 2. Indexing — accessing a character by its position
- 3. Slicing — extracting a part of a string

1. Concatenation

Concatenation means joining two or more strings using the + operator.

main.py

```
1 first_name = "Ali"  
2 last_name = "Ahmed"  
3  
4 full_name = first_name + " " + last_name  
5 print(full_name)
```

Output / Note: Ali Ahmed

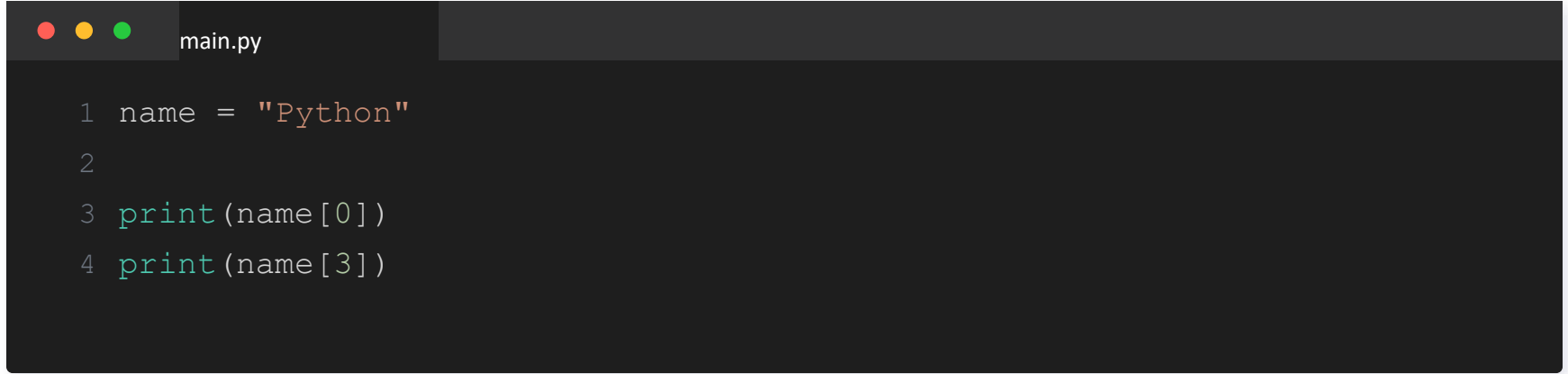
2. Indexing — Character Positions

- Every character in a string has an index number, starting from 0.

Character	P	y	t	h	o	n
Index	0	1	2	3	4	5

Accessing Characters

Use square brackets with an index number to access a character.



```
1 name = "Python"
2
3 print(name[0])
4 print(name[3])
```

Output / Note:

- P
- h

Strings are Immutable

Strings cannot be changed using indexing — this always raises an error.

main.py

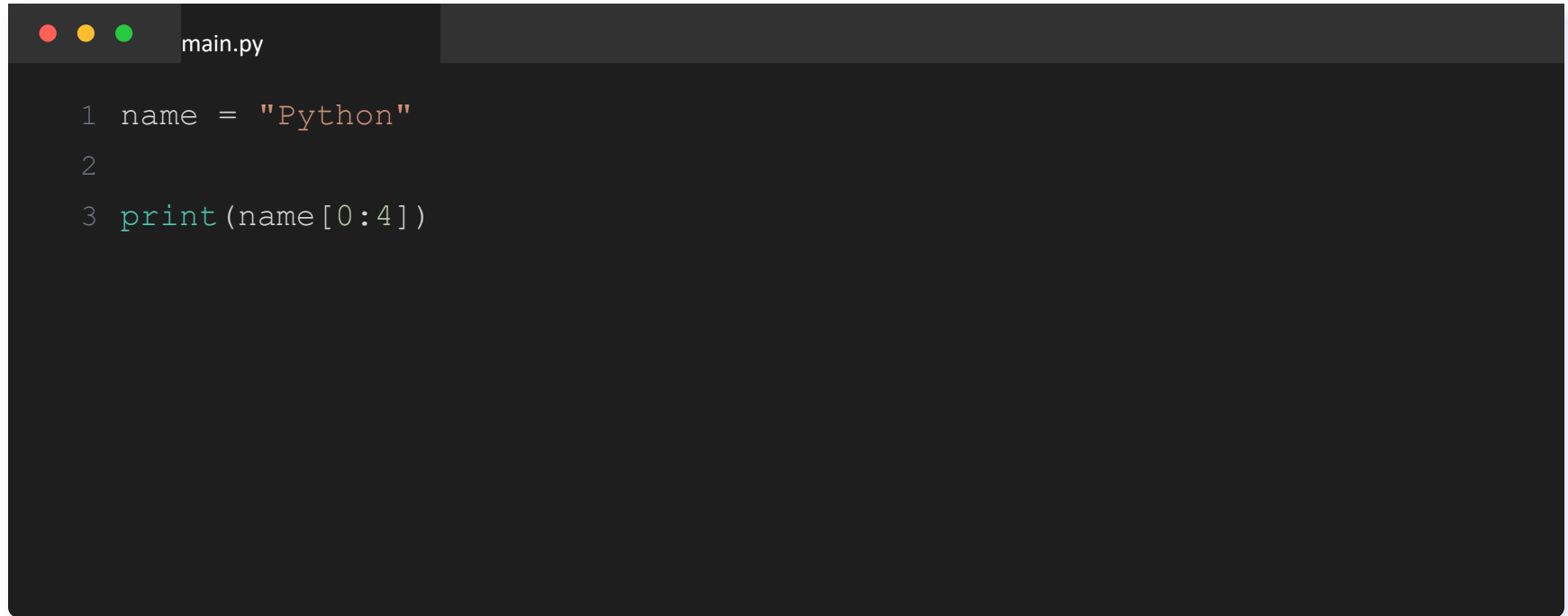
```
1 name = "Python"
2
3 name[0] = "J"
4
5 # Output: TypeError
6 # 'str' object does not support item assignment
```

3. Slicing

- Slicing is used to get a specific part of a string.
- Syntax: `string[start : end]`
- The ending index is not included in the result.

Slicing — Example

0:4 means: start from index 0 and stop before index 4.



```
1 name = "Python"
2
3 print(name[0:4])
```

Output / Note: Pyth

Slicing — Missing Indexes

If an index is left out, Python fills in the start or the end automatically.

```
main.py

1 name = "Python"
2
3 print(name[2:])      # missing end -> goes till the end
4 print(name[:4])     # missing start -> starts at index 0
```

Output / Note:

- thon
- Pyth

len() with Slicing

Using len() as the end index returns the entire string.

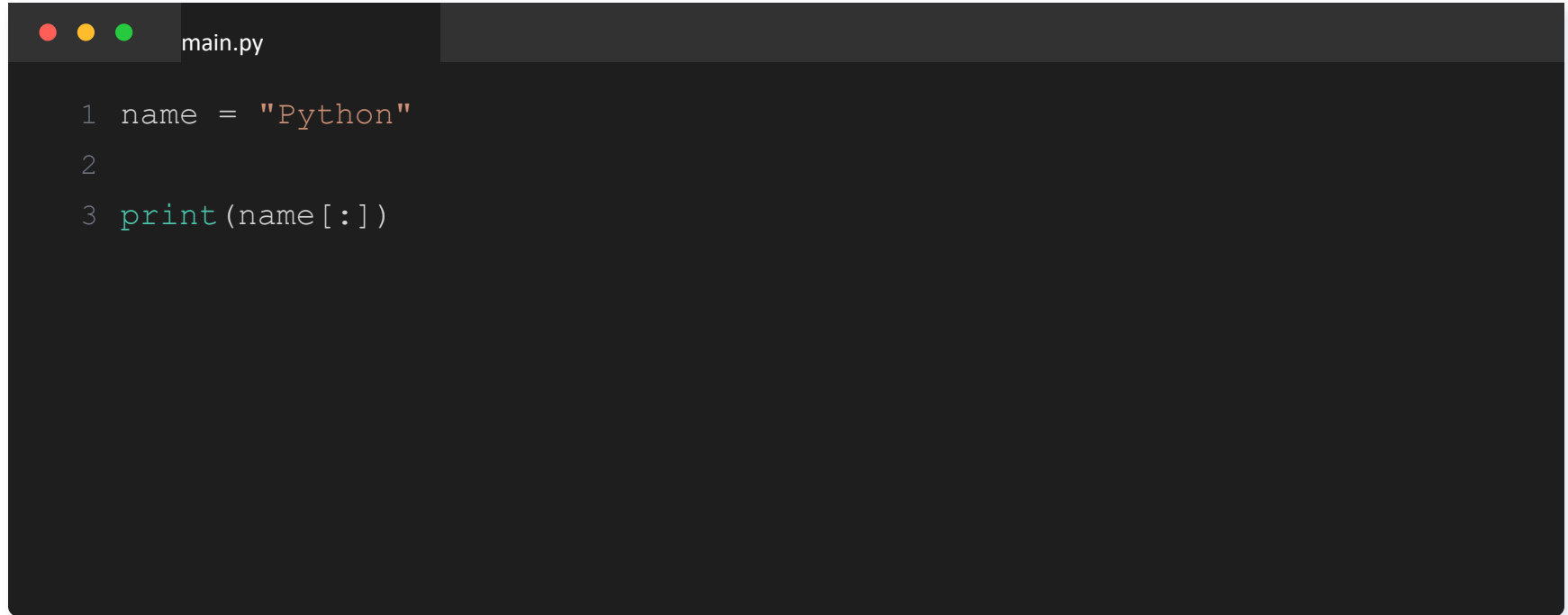
main.py

```
1 name = "Python"
2
3 print(name[0:len(name)])
```

Output / Note: Python

Copy Entire String

Leaving both start and end empty copies the whole string.



```
1 name = "Python"
2
3 print(name[:])
```

Output / Note: Python

Reverse String

A step value of -1 reverses the string using slicing.

main.py

```
1 name = "Python"  
2  
3 print(name[::-1])
```

Output / Note: nohtyP

Negative Indexing

- Negative indexing counts characters starting from the end of the string.

Character	P	y	t	h	o	n
Negative Index	-6	-5	-4	-3	-2	-1

Negative Indexing — Example

Negative indexes count backward from the last character.

main.py

```
1 name = "Python"
2
3 print(name[-1])
4 print(name[-3])
```

Output / Note:

- n
- h



String Functions / Methods

- String methods are built-in functions that perform different operations on strings.
- Most string methods return a new string and do not modify the original.
- Python provides over 20 built-in methods for case conversion, trimming, searching, splitting/joining, and type checks.

1. len()

Returns the total number of characters in a string.

main.py

```
1 text = "Python"  
2 print(len(text))
```

Output / Note: 6

2. upper() & 3. lower()

upper() converts all letters to uppercase; lower() converts them to lowercase.

main.py

```
1 text = "python"
2 print(text.upper())
3
4 text = "PYTHON"
5 print(text.lower())
```

Output / Note:

- PYTHON
- python

4. title()

Converts the first letter of each word into uppercase.

main.py

```
1 text = "python programming"  
2 print(text.title())
```

Output / Note: Python Programming

5. capitalize()

Converts only the first letter of the string into uppercase.

main.py

```
1 text = "python"  
2 print(text.capitalize())
```

Output / Note: Python

6-8. strip() / lstrip() / rstrip()

strip() removes spaces from both sides; lstrip() from the left; rstrip() from the right.

main.py

```
1 text = "  Python  "  
2  
3 print(text.strip())  
4 print(text.lstrip())  
5 print(text.rstrip())
```

Output / Note:

- Python'
- 'Python '
- ' Python'

9. replace()

Replaces one value in a string with another.

main.py

```
1 text = "I love Java"
2 print(text.replace("Java", "Python"))
```

Output / Note: I love Python

10. find() & 11. rfind()

find() returns the first index of a value; rfind() returns the last occurrence.

main.py

```
1 text = "I love Python"
2 print(text.find("Python"))
3
4 text = "apple apple"
5 print(text.rfind("apple"))
```

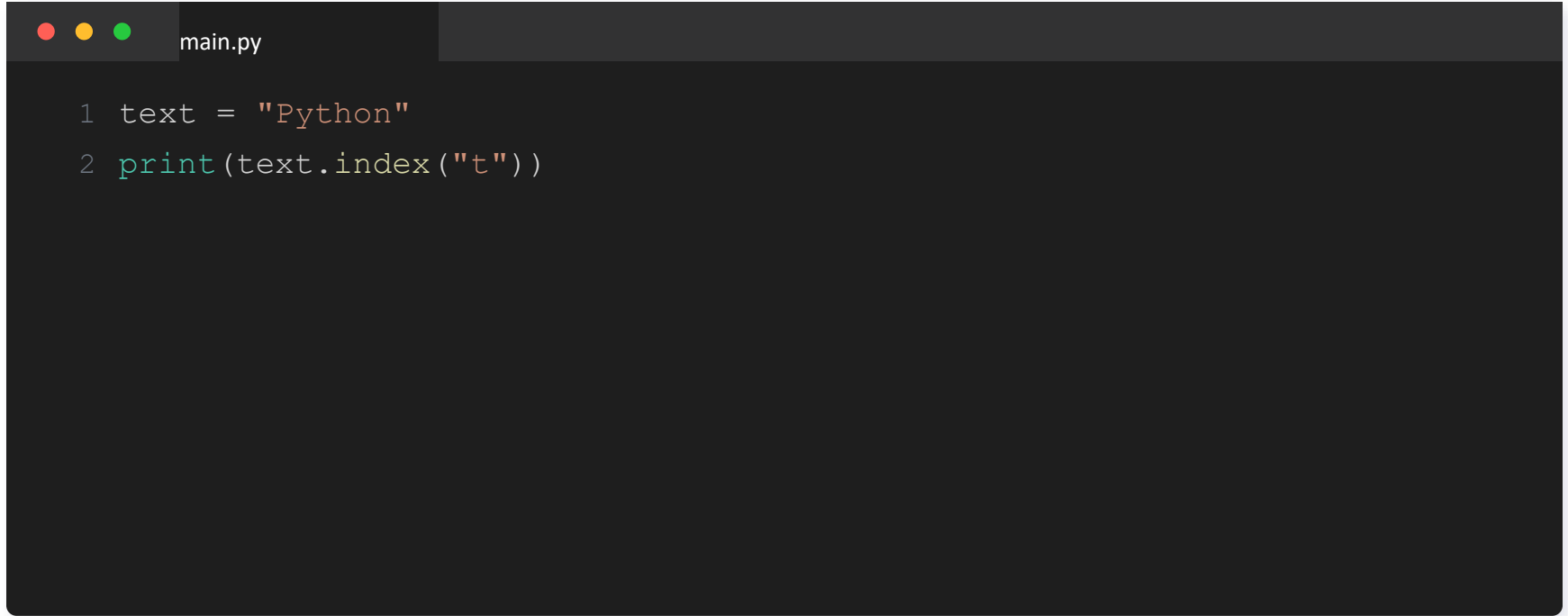
Output / Note:

- 7
- 6

(find returns -1 if not found)

12. index()

Works like find(), but raises an error if the value is not found.



```
1 text = "Python"
2 print(text.index("t"))
```

Output / Note: 2

13. count()

Counts how many times a value appears in a string.



main.py

```
1 text = "apple apple mango apple"
2 print(text.count("apple"))
```

Output / Note: 3

14. startswith() & 15. endswith()

Check whether a string starts or ends with a given value.

main.py

```
1 text = "Python"
2 print(text.startswith("Py"))
3
4 text = "notes.pdf"
5 print(text.endswith(".pdf"))
```

Output / Note:

- True
- True

16. split()

Splits a string into a list, using spaces by default.

main.py

```
1 text = "Python Java C++"  
2 print(text.split())
```

Output / Note: ['Python', 'Java', 'C++']

17. join()

Joins list elements into a single string using a separator.

main.py

```
1 languages = ["Python", "Java", "C++"]  
2 print(", ".join(languages))
```

Output / Note: Python, Java, C++

18-20. isalpha() / isdigit() / isalnum()

Check whether a string contains only letters, only digits, or letters+numbers.

main.py

```
1 text = "Python"
2 print(text.isalpha())
3
4 text = "12345"
5 print(text.isdigit())
6
7 text = "Python123"
8 print(text.isalnum())
```

Output / Note:

- True
- True
- True

21-22. islower() & isupper()

Check whether all letters in a string are lowercase or uppercase.

main.py

```
1 text = "python"
2 print(text.islower())
3
4 text = "PYTHON"
5 print(text.isupper())
```

Output / Note:

- True
- True

23. swapcase()

Converts uppercase letters to lowercase and lowercase letters to uppercase.

main.py

```
1 text = "PyThOn"  
2 print(text.swapcase())
```

Output / Note: pYtHoN

24. center()

Centers the string within a specified width, padding with spaces.

main.py

```
1 text = "Python"  
2 print(text.center(20))
```

Output / Note: ' Python ' (centered in 20 characters)

Important Points

- Strings are enclosed in single, double, or triple quotes.
- Strings are immutable (cannot be modified directly).
- Indexing starts from 0.
- Negative indexing starts from -1.
- Slicing extracts a part of a string.
- Most string methods return a new string and do not modify the original string.
- Strings support many built-in methods for searching, formatting, checking, and manipulating text.

Strings — Summary Table

Category	Topics
Quotes	Single Quotes, Double Quotes, Triple Quotes
String Operations	Concatenation, Indexing, Slicing
Common String Methods	<code>len()</code> , <code>upper()</code> , <code>lower()</code> , <code>title()</code> , <code>capitalize()</code> , <code>strip()</code> , <code>lstrip()</code> , <code>rstrip()</code> , <code>replace()</code> , <code>find()</code> , <code>rfind()</code> , <code>index()</code> , <code>count()</code> , <code>startswith()</code> , <code>endswith()</code> , <code>split()</code> , <code>join()</code> , <code>isalpha()</code> , <code>isdigit()</code> , <code>isalnum()</code> , <code>islower()</code> , <code>isupper()</code> , <code>swapcase()</code> , <code>center()</code>

Strings (1/3) — Multiple Choice Questions

1. Which type of quotes is best for multiline strings?

A) Single B) Double C) Triple D) None

2. Which operator is used for string concatenation?

A) + B) - C) * D) &

3. What does name[0] return for name = "Python"?

A) n B) P C) y D) o

4. Strings in Python are:

A) Mutable B) Immutable C) Numbers D) Lists

5. What does name[:] return?

A) An empty string B) The entire string C) Only the first character
D) An error

Answer Key: 1-C 2-A 3-B 4-B 5-B

Strings (2/3) — Multiple Choice Questions

1. What does `name[::-1]` do?

- A) Copies the string B) Reverses the string C) Deletes the string
D) Capitalizes the string

2. What does `name[-1]` return?

- A) The first character B) The last character C) An error D)
Nothing

3. What does `name[2:]` return for "Python"?

- A) Py B) thon C) Python D) on

4. What does `name[:4]` return for "Python"?

- A) Pyth B) thon C) Python D) hon

5. `print(name[0:len(name)])` prints:

- A) Nothing B) An error C) The entire string D) Only index 0

Answer Key: 1-B 2-B 3-B 4-A 5-C

Strings (3/3) — Multiple Choice Questions

1. Which method converts the first letter of each word to uppercase?

A) capitalize() B) title() C) upper() D) swapcase()

2. Which method removes spaces from both sides of a string?

A) strip() B) lstrip() C)rstrip() D) trim()

3. Which method returns -1 if a value is not found?

A) index() B) find() C) count() D) search()

4. Which method raises an error if the value is not found?

A) find() B) index() C) count() D) rfind()

5. Which method checks if a string contains only digits?

A) isalpha() B) isalnum() C) isdigit() D) isupper()

Answer Key: 1-B 2-A 3-B 4-B 5-C

Python Operators

- Operators are special symbols used to perform operations on variables and values.
- Syntax: `variable = value1 operator value2`

Operators — Basic Example

An operator combines two values and produces a result.

main.py

```
1 a = 10
2 b = 5
3
4 result = a + b
5 print(result)
```

Output / Note: 15

1. Arithmetic Operators

- Arithmetic operators are used to perform mathematical calculations.

Operator	Name	Description	Example
+	Addition	Adds two values	$10 + 5 = 15$
-	Subtraction	Subtracts one value from another	$10 - 5 = 5$
*	Multiplication	Multiplies two values	$10 * 5 = 50$
/	Division	Returns floating-point division	$10 / 5 = 2.0$
//	Floor Division	Returns integer quotient	$10 // 3 = 3$
%	Modulus	Returns remainder	$10 \% 3 = 1$
**	Exponent	Raises power	$2 ** 3 = 8$

Addition (+)

Syntax: `result = a + b`

main.py

```
1 a = 20
2 b = 15
3 print(a + b)
```

Output / Note: 35

Subtraction (-)

Syntax: `result = a - b`

main.py

```
1 a = 20
2 b = 8
3 print(a - b)
```

Output / Note: 12

Multiplication (*)

Syntax: `result = a * b`

main.py

```
1 a = 6
2 b = 7
3 print(a * b)
```

Output / Note: 42

Division (/)

Syntax: `result = a / b`

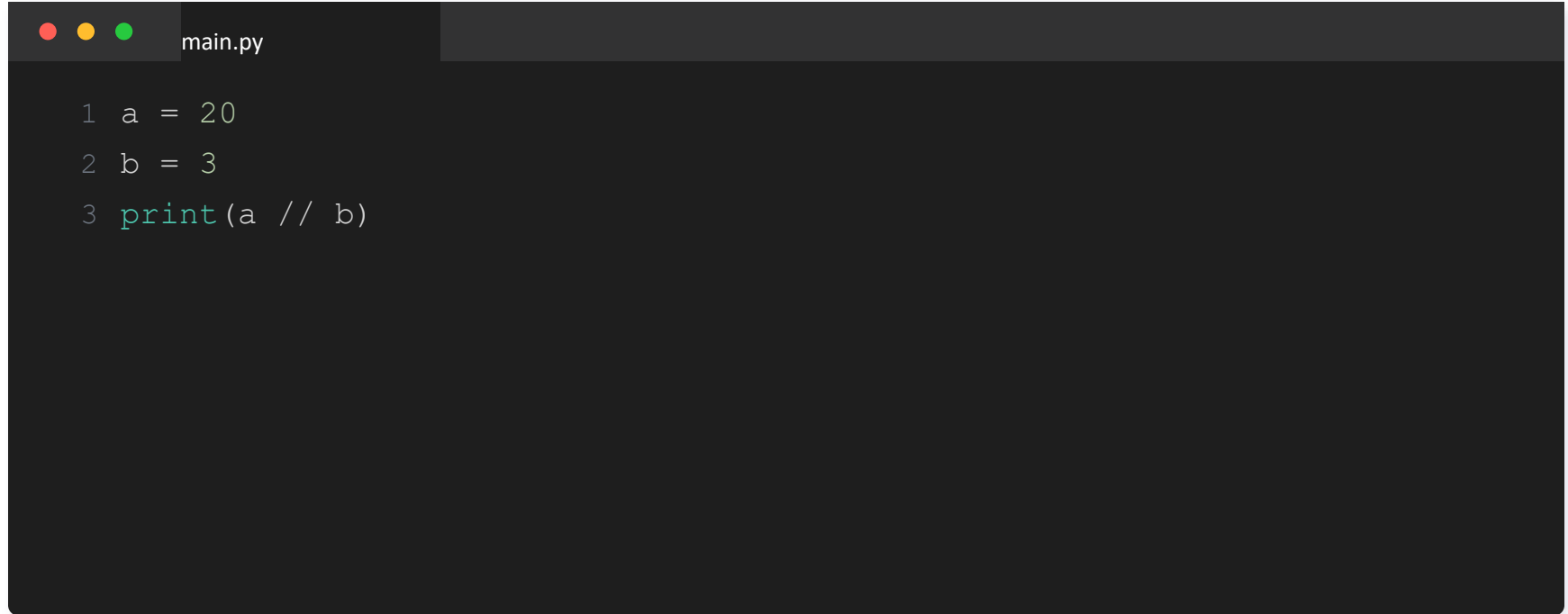
main.py

```
1 a = 20
2 b = 4
3 print(a / b)
```

Output / Note: 5.0

Floor Division (//)

Syntax: `result = a // b`



```
1 a = 20
2 b = 3
3 print(a // b)
```

Output / Note: 6

Modulus (%)

Syntax: `result = a % b`

main.py

```
1 a = 20
2 b = 3
3 print(a % b)
```

Output / Note: 2

Exponent (**)

Syntax: `result = a ** b`

main.py

```
1 a = 2
2 b = 5
3 print(a ** b)
```

Output / Note: 32

2. Assignment Operators

- Assignment operators are used to assign values to variables.

Operator	Name	Example	Equivalent To
=	Assignment	x = 5	x = 5
+=	Add and Assign	x += 3	x = x + 3
-=	Subtract and Assign	x -= 3	x = x - 3
*=	Multiply and Assign	x *= 3	x = x * 3
/=	Divide and Assign	x /= 3	x = x / 3
//=	Floor Divide and Assign	x //= 3	x = x // 3
%=	Modulus and Assign	x %= 3	x = x % 3
**=	Power and Assign	x **= 2	x = x ** 2

Assignment Operators — Example

Assignment operators update a variable's value in place.

main.py

```
1 x = 10
2 x += 5
3 print(x)
4
5 x *= 2
6 print(x)
7
8 x -= 4
9 print(x)
```

Output / Note:

- 15
- 30
- 26

3. Comparison (Relational) Operators

- Comparison operators compare two values and always return True or False.

Operator	Meaning	Example
==	Equal to	5 == 5
!=	Not equal to	5 != 3
>	Greater than	5 > 3
<	Less than	5 < 3
>=	Greater than or equal	5 >= 5
<=	Less than or equal	5 <= 6

Comparison Operators — Example

Comparison operators are commonly used inside conditions.

main.py

```
1 a = 15
2 b = 10
3 print(a == b)
4 print(a != b)
5 print(a > b)
6 print(a < b)
7 print(a >= b)
8 print(a <= b)
```

Output / Note:

- False
- True
- True
- False
- True
- False

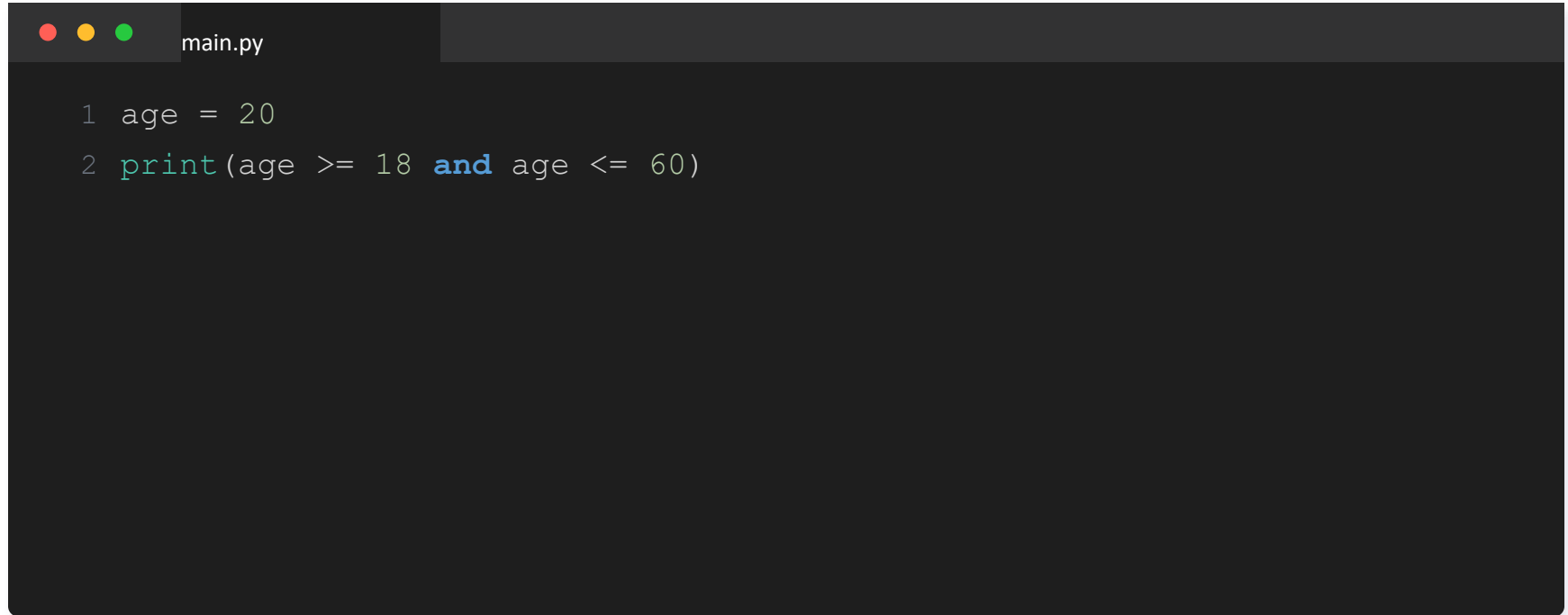
4. Logical Operators

- Logical operators combine multiple conditions.

Operator	Meaning
and	True if both conditions are True
or	True if at least one condition is True
not	Reverses the result

Logical Operator: and

and returns True only if both conditions are True.

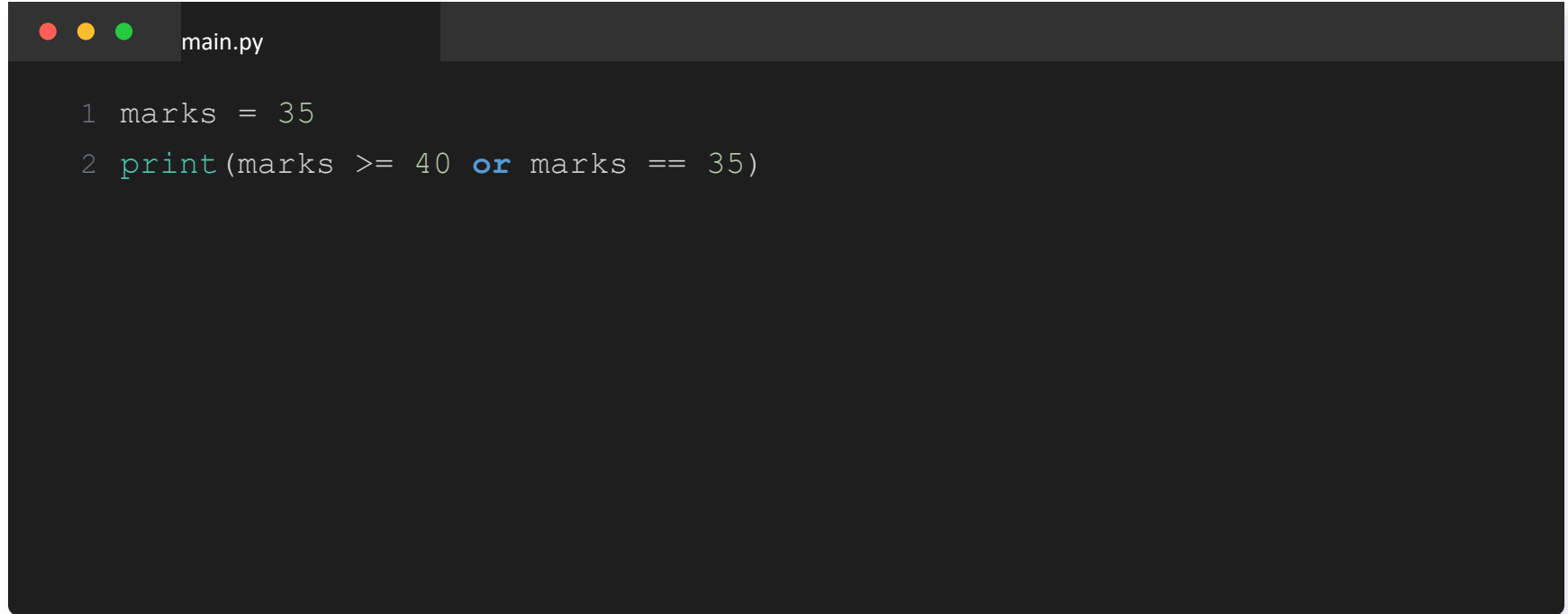


```
main.py
1 age = 20
2 print(age >= 18 and age <= 60)
```

Output / Note: True

Logical Operator: or

or returns True if at least one condition is True.



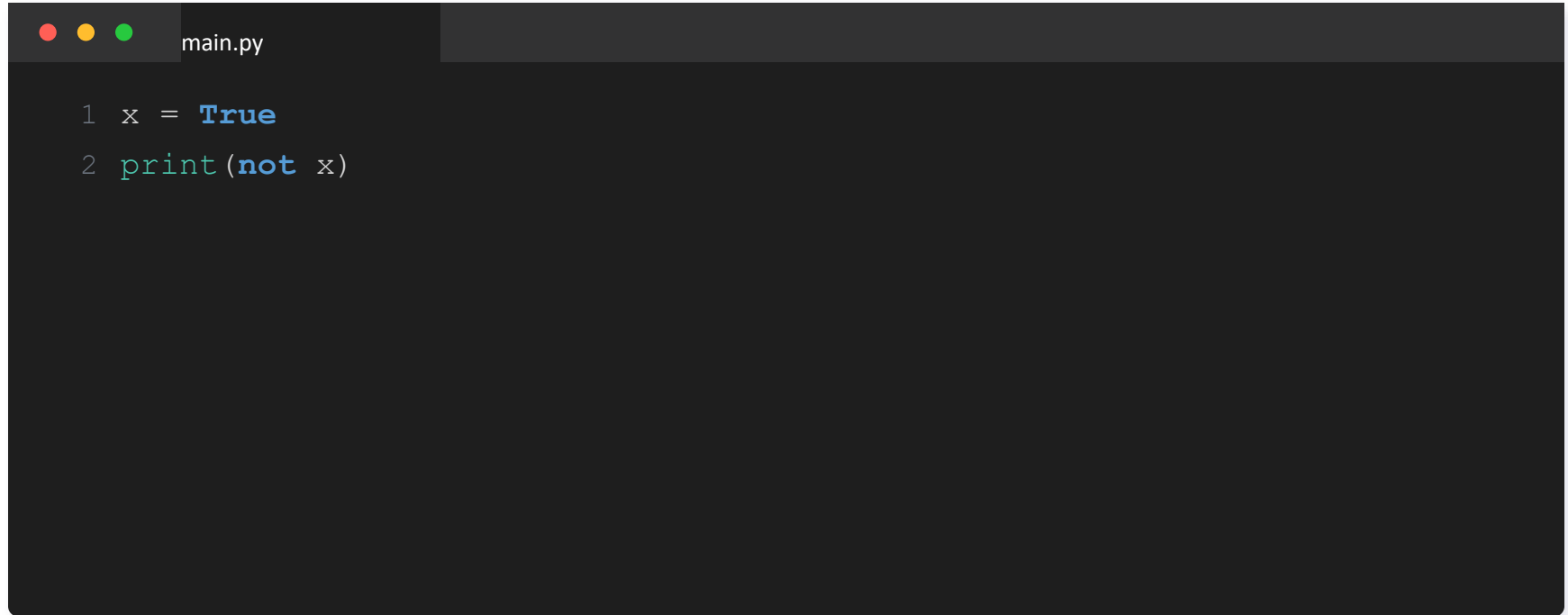
```
main.py

1 marks = 35
2 print(marks >= 40 or marks == 35)
```

Output / Note: True

Logical Operator: not

not reverses the result of a condition.



```
1 x = True
2 print(not x)
```

Output / Note: False

5. Identity Operators

- Identity operators check whether two variables refer to the same object in memory.

Operator	Meaning
is	Same object
is not	Different objects

Identity Operators — Example

is compares object identity, not just equal values.

```
main.py
1 a = [1, 2]
2 b = a
3 c = [1, 2]
4
5 print(a is b)
6 print(a is c)
7 print(a is not c)
```

Output / Note:

- True
- False
- True

6. Membership Operators

- Membership operators check whether a value exists in a sequence.

Operator	Meaning
in	Value exists
not in	Value does not exist

Membership Operators — Example

`in` and `not in` check whether a value exists inside a sequence.

main.py

```
1 fruits = ["Apple", "Banana", "Orange"]
2
3 print("Apple" in fruits)
4 print("Mango" in fruits)
5 print("Mango" not in fruits)
```

Output / Note:

- True
- False
- True

Operators (1/2) — Multiple Choice Questions

1. Which operator returns the remainder of a division?

A) / B) // C) % D) **

2. What does `10 // 3` return?

A) 3.33 B) 3 C) 1 D) 4

3. Which operator is used for exponentiation?

A) ^ B) ** C) %% D) //

4. `x += 3` is equivalent to:

A) `x = 3` B) `x = x + 3` C) `x = x - 3` D) `x == 3`

5. Which operator always returns True or False?

A) Arithmetic B) Assignment C) Comparison D) Membership only

Answer Key: 1-C 2-B 3-B 4-B 5-C

Operators (2/2) — Multiple Choice Questions

1. Which logical operator reverses a result?

A) and B) or C) not D) is

2. and returns True when:

A) Either condition is True B) Both conditions are True C) Neither is True D) Always

3. Which operator checks if two variables point to the same object?

A) == B) is C) in D) =

4. Which operator checks membership in a sequence?

A) is B) in C) not D) ==

5. "Mango" not in fruits returns True when:

A) Mango exists in fruits B) Mango does not exist in fruits C) fruits is empty only D) Always False

Answer Key: 1-C 2-B 3-B 4-B 5-B



Thank You!

Keep practicing — write code every day to master Python.